scanning, automatic service registration, and property injection. Replacing the built-in container can also be useful when you're porting an existing app that uses a third-party DI container to ASP.NET Core.

The .NET community had used DI containers for years before ASP.NET Core decided to include a built-in one. The ASP.NET Core team wanted a way to use DI in their own framework libraries, and they wanted to create a common abstraction[1] that allows you to replace the built-in container with your favorite third-party alternative, such as Autofac, StructureMap/Lamar, Ninject, Simple Injector, or Unity.

The built-in container is intentionally limited in the features it provides, and realistically, it won't be getting many more. By contrast, third-party containers can provide a host of extra features. These are some of the features available in Lamar (https://jasperfx.github.io/lamar/guide/ioc), the spiritual successor to StructureMap (https://structuremap.github.io):

- Assembly scanning for interface/implementation pairs based on conventions
- Automatic concrete class registration
- Property injection and constructor selection
- Automatic `Lazy<T>`/`Func<T>` resolution
- Debugging/testing tools for viewing inside your container

None of these features is a requirement for getting an application up and running, so using the built-in container makes a lot of sense if you're building a small app or are new to DI containers in general. But if at some undefined tipping point, the simplicity of the built-in container becomes too much of a burden, it may be worth replacing.